

Architecture of the Rust NDB API

Threads, the data model, and two ways to program against it: asynchronous
and synchronous

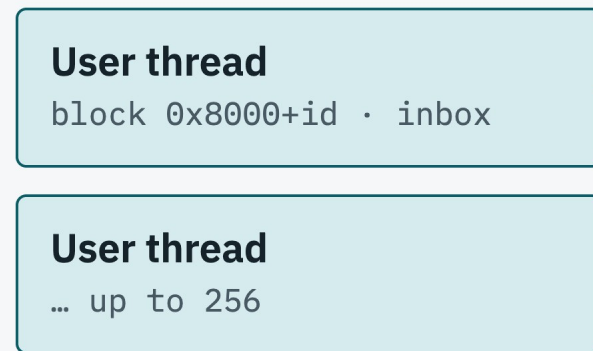
One rule shapes everything

Receive threads **receive and route**.
User threads **execute**.

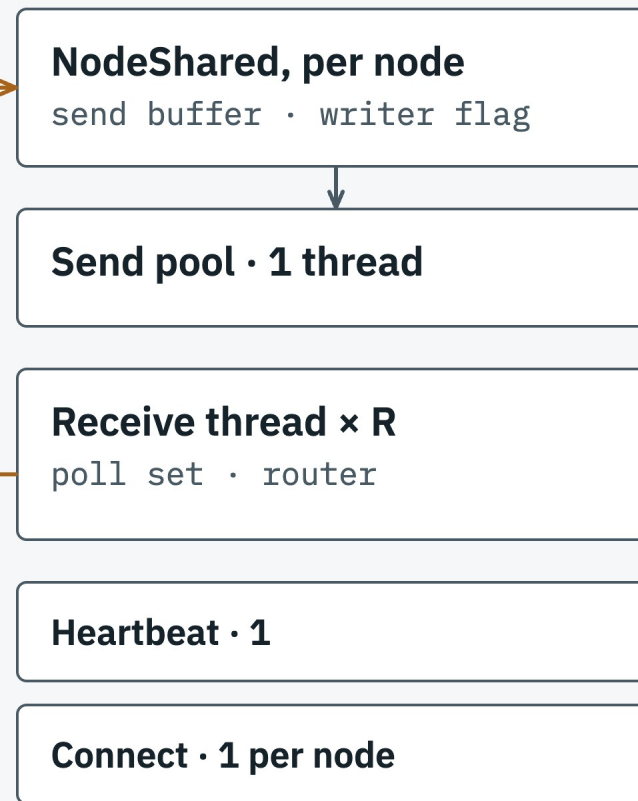
The C++ NDB API decodes every signal on its receive thread, which becomes the bottleneck with many threads. Here a receive thread does one lookup and one copy per signal; decoding, state and callbacks run on the thread that owns the query.

The threads today

APPLICATION = USER THREADS



SHARED



RONDB



send

1 write

1 page / round

replies

What keeps it fast

A node's state has one writer

The receive thread that owns a node executes its heartbeat replies. Everyone else reads atomics; no mutex.

One lock per inbox per round

Signals are sorted per thread with no lock held, then posted as one page under one short lock.

Wake a thread only if it sleeps

A waiting flag under the inbox mutex: posting to a busy thread costs no system call.

The thread that sends does the write

Append to the node's send buffer; if no one is writing to that node, take the buffer and write it outside the mutex. Later signals go to the pool.

What it does today

Setup (primary key reads)	Reads / s	Limited by
Mac, 1 user thread, depth 4	2.6 M	The user thread: ~350 ns user time per read
Mac, 4 user threads, batch 300	5.8 M	Cores shared with the data nodes
Mac, 8 user threads, batch 300	7.0–8.9 M	One data node TC thread at 96%
Linux, 8 user threads, 2 receive threads	7.1 M	The data node's receive thread

Client CPU stays near **500–550 ns per read** from depth 1 to 4 and from 1 to 16 threads. The limits are now in the data nodes and in shared cores. `tools/ic_model` reproduces these to within 14%.

How it evolves, in order of payoff

1	Wake-up descriptor in the poll set A self-pipe or eventfd: new links watched at once, no 20 ms idle wake-ups.	Planned	2	Executor threads for applications A second list in the inbox. Any number of application threads, few executors.	Proposed
3	More user threads 256 → 4711 per process, for a thread per client connection.	Planned	4	Wake-up helpers, larger send pool When a round wakes hundreds of threads, helpers do the wake-ups.	Planned
5	Nodes placed by load Move a node between receive threads without breaking the one-writer rule.	Planned			

Everything a transaction needs lives in one thread

Shared: ApidGlobal

NodeShared per node: published state (atomics), membership word (CAS), and the Sender: waiting and spare buffers, writer flag, adaptive send. Thread table, send pool, dictionary cache.

Per thread: ApidConnection

Inbox, queries and transactions in generational maps, TC records with free lists, the active vector, per-node staged and outgoing buffers, executed list.

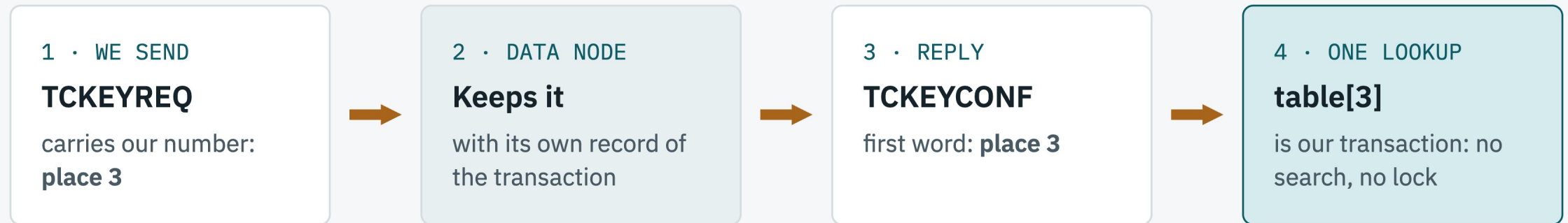
ApidQuery and Transaction

A query owns its key and attribute rows: no pointer into application memory, no unsafe. A transaction is local until its first send.

Query: Idle → Defined → Sent → Completed → Idle · no allocation per transaction once warm

How a reply finds its transaction

Each request carries **a number we choose**, and the data node sends it back in its reply. We choose it to be the transaction's **place in our own table**, so one lookup finds it.



Rows work the same way. A row (TRANSID_AI) carries the query's id instead, and finds the query the same way.

Late replies are caught. Every reply also carries the transaction id; one left over from an earlier use of place 3 does not match, and is dropped.

Pack at define, patch at send; pages go round

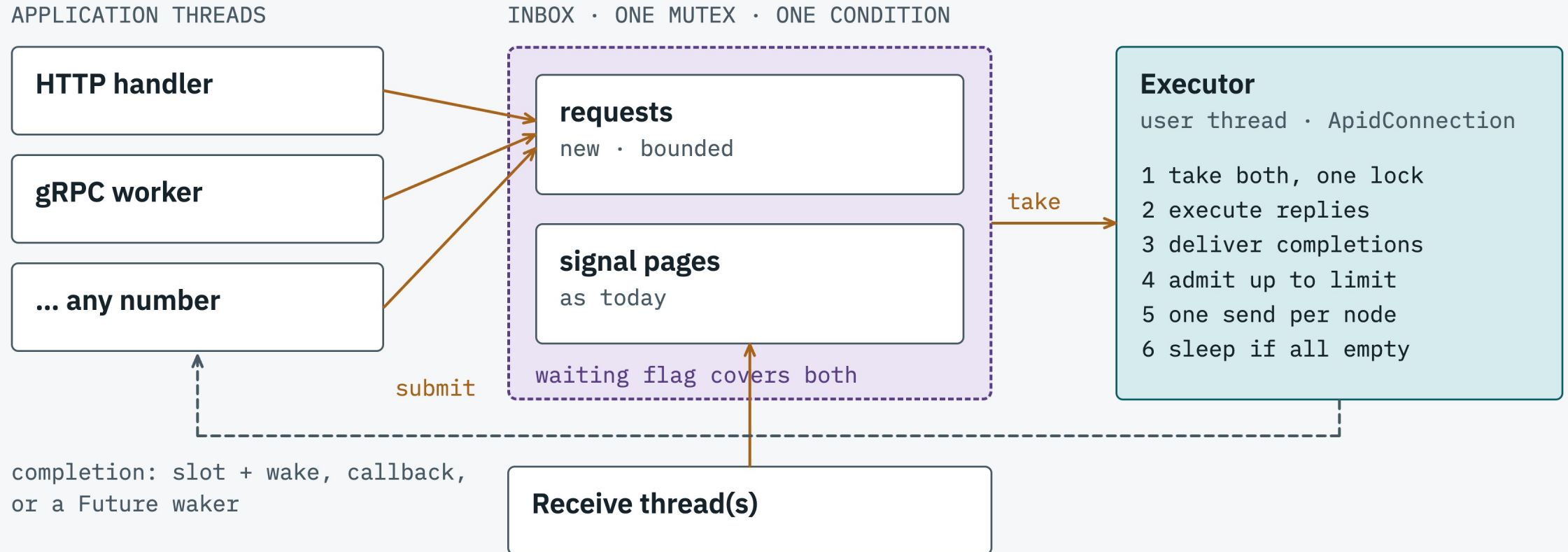
SEND

- a Define**
TCKEYREQ packed straight into the node's staged buffer
- b Send**
Start, execute and commit flags set in place; staged moves to outgoing
- c Write**
Append to the node's send buffer; if no one else is writing, take it and write: one socket write per node

RECEIVE

- a Read**
Into 128 KB receive pages, counted by Arc, reused last in first out
- b Route**
Under 256 words: copied into the thread's page; larger: by reference
- c Poll**
Swap emptied pages for posted ones; read every signal in place

An executor listens to two inputs



One wait point, one loop

```
fn submit(&self, r: Request) {  
    let wake = {  
        let mut ib = self.inbox.lock();  
        ib.requests.push(r);  
        take(&mut ib.waiting)  
    };  
    if wake { self.cond.signal(); }  
}
```

Same shape as a receive thread's post: append under a short lock, wake only if asleep.

Replies before requests: they free records and room in the pipeline.

Batching comes on its own: all requests admitted in one pass share one write per node.

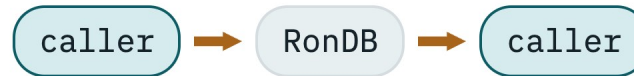
Back-pressure twice: a bounded request list, and an in-flight limit.

The caller waits for its answer: two ways

Built

A • Talk to RonDB directly

The application thread owns a connection, sends, and waits for its own reply.



Good: nothing in between, the lowest latency

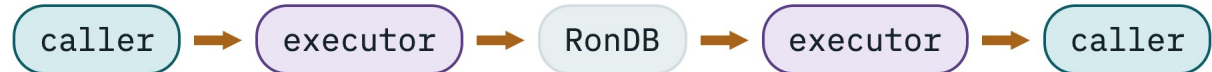
Cost: one call at a time per thread

Cost: a connection per thread: 256 today, 4711 planned

Proposed

B • Hand the call to an executor

The thread submits the call and sleeps; an executor sends it and wakes the thread with the answer.



Good: any number of callers, few executors

Good: calls from many callers share one send per node

Cost: one hand-off each way, about 8 μ s if the caller sleeps

SUMMARY

One addition opens both models

A second list in the inbox, under the **same lock and wake rule, lets any thread give work to an executor.**

Next: the executor, then blocking calls and Futures on top of it.

Read more: <doc/rust/architecture.html>, chapters 02, 04, 06 and 12.